

# LegalEase AI - Backend Architecture & Services

---

## Overview

---

The backend of **LegalEase AI** is built on **FastAPI** ( `backend/main.py` ), utilizing **Uvicorn** as the ASGI server. It implements Pydantic models for validation, orchestrates Google Gemini AI for document drafting, and utilizes specialized document generation services for file rendering.

## Directory Organization

---

```
text
backend/
├── __init__.py
├── main.py                # Application factory, middleware & exception handlers
├── config.py              # Environment settings & constants
├── api/
│   ├── __init__.py
│   ├── routes.py          # Document generation endpoints
│   └── health.py          # Root and service health diagnostics
├── models/
│   ├── __init__.py
│   └── schemas.py         # Pydantic request & response models
├── ai_core/
│   ├── __init__.py
│   └── gemini_generator.py # Google Gemini SDK orchestration & prompt logic
├── services/
│   ├── __init__.py
│   ├── document_service.py # Unified document generation service
│   ├── txt_generator.py    # Plain text formatter
│   ├── docx_generator.py  # python-docx Microsoft Word renderer
│   └── pdf_generator.py    # ReportLab PDF renderer
└── utils/
    ├── __init__.py
    ├── validation.py       # Domain validation rules
    ├── sanitization.py     # Text sanitization & terms parsing
    └── error_handler.py    # Safe custom exception handlers
```

# Core Modules & Responsibilities

---

## 1. Application Entrypoint ( `main.py` )

- Initializes `FastAPI(title="LegalEase - AI Legal Document Generator")`.
- Registers CORS middleware to enable secure communication with frontend clients.
- Registers global and domain exception handlers ( `LegalEaseException` , `Exception` ).
- Mounts routers: `health_router` and `generate_router`.

## 2. Configuration ( `config.py` )

- Reads environment variables via `python-dotenv` :
  - `GEMINI_API_KEY` : API key for Gemini models.
  - `GEMINI_MODEL` : Model identifier (default: `gemini-2.5-flash` ).
  - `BACKEND_HOST` , `BACKEND_PORT` , `BACKEND_URL` .
- Defines application constants, supported document categories, and the mandatory legal disclaimer.

## 3. Pydantic Schemas ( `schemas.py` )

- `DocumentRequest` :
  - `document_type` : Non-empty string.
  - `parties` : Mandatory description of parties.
  - `terms` : Accepts `List[str]` or semicolon-separated `str` , normalized into a list of strings.
  - `agreement_date` , `effective_date` : Required date strings.
  - `start_date` , `end_date` : Optional dates with cross-validation ensuring `end_date >= start_date` .
- `DocumentResponse` :
  - `success` : Boolean status.
  - `document_type` : Echo of document category.
  - `content` : Sanitized legal document text.
  - `terms_count` : Number of parsed terms.
- `HealthResponse` :
  - `status` : Service status string.
  - `model_configured` : Active Gemini model name.

- `api_key_configured` : Boolean indicator whether an API key is present.

#### 4. AI Core ( `gemini_generator.py` )

- Orchestrates Google GenAI clients (supporting both the modern `google-genai` and `google-generativeai` SDKs).
- Builds comprehensive prompts enforcing legal drafting standards.
- Fallback mechanism across models ( `gemini-2.5-flash` , `gemini-1.5-flash` , `gemini-1.5-pro` ).
- Sanitizes AI responses to strip markdown fences and normalize special characters.

#### 5. Document Service Layer ( `services/` )

- `document_service.py` : Coordinates AI generation, HTML previews, and binary exports.
- `txt_generator.py` : Exports formatted plain text with ASCII dividers and legal disclaimers.
- `docx_generator.py` : Builds Microsoft Word `.docx` documents with:
  - Header with embedded organization logo.
  - Times New Roman typography.
  - Numbered section headings and indented clauses.
  - Shaded Schedule A table for key terms.
  - Signature lines and date blanks.
  - Running footer with copyright and legal disclaimer.
- `pdf_generator.py` : Uses ReportLab to generate multi-page PDFs with:
  - Custom `NumberedCanvas` computing "Page X of Y".
  - Running header and footer rules.
  - Branded logo centered at the top.
  - Auto-wrapped paragraphs, bullet lists, and terms table.
  - Multi-party execution signature blocks.

#### 6. Safe Error Handling ( `error_handler.py` )

- Catches known domain errors ( `MissingAPIKeyError` , `GeminiAPIError` , `DocumentGenerationError` ).
- Ensures that internal server traces, stack traces, and API credentials are never leaked in HTTP response bodies.